Batch 18 – Analysis sheet: nightly batch jobs with deadlines

Unit jobs, 1-hour slots, profit only if on time. Chosen: **Greedy (profit descending) + DSU for the latest free slot**. Rejected (implemented): **Backtracking**. Load: $n = 10^6$ jobs, $D = 10^4$ slots, < 2 s.

1. Results on the 8-job instance (Rs thousand)

Method	Schedule (hour 1–4)	Profit	Comment
Greedy + DSU (optimal)	J8, J1, J7, J6	**230**	Rejected J3, J4, J2, J5. Upper bound: 4 slots, top-4 profits 100+60+40+30.
EDF, ties by input order	J2, J1, J5, J6	194	Ignores profit: J2(19) blocks J4(25); J5(15) blocks J7(40).
EDF, ties by higher profit	J4, J1, J7, J6	225	Best case for EDF, still loses 5 (J8 never scheduled).
Backtracking	(same set as greedy)	230	280 search nodes on 8 jobs; matches greedy on all 10 test inputs.

2. Data structures for "latest free slot $\le d$" ($n = 10^6$, $D = 10^4$)

Structure	Per job	Space	Total find cost	Measured find (random / worst)	Meets 2 s?
Linear scan	$O(D)$	$O(D)$	$\le nD = 10^{10}$ probes (4.95×10^9 / 9.95×10^9 measured)	0.74 s / 2.18 s	worst: no
Balanced BST (`TreeSet`)	$O(\log D)$	$O(D)$	$n \log_2 D \approx 1.3 \times 10^7$	15 ms / 5.5 ms	yes
Segment tree	$O(\log D)$	$O(D)$	$\approx 2n \log_2 D \approx 2.7 \times 10^7$	3.9 ms / 2.1 ms	yes
DSU + path compression	$\approx O(\alpha(D))$	$O(D)$	$\approx n\alpha \approx 10^6$ hops (1.02×10^6 / 1.01×10^6 measured)	2.6 ms / 1.6 ms	yes

Common sort by profit: $n \log_2 n \approx 2.0 \times 10^7$ comparisons, 76 ms. End-to-end Greedy + DSU: 93 ms at 10^6 jobs.

3. Technique comparison

Technique	Time	Optimal?	Fits 10^6 jobs?	Reason / example
Greedy + DSU	$O(n \log n)$	yes (proved)	yes, 93 ms	Matroid: exchange argument, greedy-choice property holds.
Brute force	$O(2^n)$ or $O(n!)$	yes	no	$n = 30$: 10^9 subsets.
Divide & conquer	–	not as split	no	Halves compete for slots; deadline split gives 227 $<$ 230.
Dynamic programming	$O(nD)$	yes	no (10^{10} updates)	Needed for variable durations, redundant for unit jobs.
Backtracking	$O(2^n D)$	yes	no	$n = 30$: $2^{31} - 1$ nodes, 6.3 s; $n = 40 \approx 1.8$ h.

4. Empirical timing (median ms, Java 21, 2 cores): both algorithms on the same inputs

n	10	15	20	25	30	Growth
Greedy + DSU (random)	0.001	0.002	0.004	0.004	0.006	about linear
Backtracking (random)	0.004	0.098	3.19	76.3	2258	about $\times30$ from $n = 25$ to 30: exponential
Backtracking (worst: all $d = n$)	0.006	0.188	6.20	186	6342	$\times2$ per extra job ($2^{n+1} - 1$ nodes)

Greedy + DSU alone: $n = 10^3, 10^4, 10^5, 5 \cdot 10^5, 10^6$ takes 0.07, 0.9, 7.5, 53, 93 ms (random); at 10^6: 93 ms random, 92 ms same-deadline, 6.6 ms chain.

5. Complexity of Greedy + DSU and the changed requirement

Time $O(n \log n + (n + D)\alpha(D)) = O(n \log n)$; best $O(n + D)$ with counting sort; worst: a chain input makes one find walk D nodes, paid for once by compression. **Space** $O(n + D)$.

Durations 1–4 h (Q5). Example A=(3 h, d=3, profit 10), B=(1 h, d=1, 6), C=(2 h, d=3, 6): greedy takes A $\Rightarrow$ 10, optimum B+C $=$ 12. Use DP over time (Lawler–Moore): $O(nD)$ time, $O(D)$ space; 10^{10} updates ≈ 6.5 s at $n = 10^6$ (measured 645 ms at $n = 10^5$), so it needs parallelism, pruning or approximation to meet 2 s.